

Переменные в Transact SQL.

Локальная переменная Transact-SQL представляет собой объект, содержащий некоторое значение определенного типа. Переменные обычно используются в сценариях и для хранения временных данных:

- в качестве счетчика цикла;
- для хранения значения, которое необходимо проверить инструкцией управления потоком;
- для хранения значения, возвращенного функцией или хранимой процедурой.

Имена локальных переменных Transact-SQL начинаются с одного символа «@».

Инструкция **DECLARE** инициализирует переменную Transact-SQL следующим образом:

- Назначение имени. Первым символом имени должен быть одиночный символ @.
- Назначение длины и типа данных, определяемого системой или пользователем. Для числовых переменных задаются также точность и масштаб. Для переменных типа XML может быть дополнительно задана коллекция схем.
- Присваивает созданной переменной значение NULL.

Например, следующая инструкция DECLARE создает локальную переменную @mycounter типа данных int:

```
DECLARE @MyCounter int
```

Инструкция DECLARE позволяет объявить несколько локальных переменных одинакового или разного типов через запятую. Например, следующая инструкция DECLARE создает три локальные переменные, @LastName, @FirstName и @StateProvince, и присваивает каждой из них значение NULL:

```
DECLARE @LastName nvarchar(30),  
        @FirstName nvarchar(20),  
        @StateProvince nchar(2)
```

Объявление переменных должно быть осуществлено в той транзакции, в которой выполняется команда, использующая эту переменную. Иначе говоря, после завершения транзакции, то есть после команды GO, переменная уничтожается.

При объявлении переменной присваивается значение NULL. Чтобы изменить значение переменной, применяется инструкция SET. Этот способ присвоения значений переменным является предпочтительным. Кроме того, переменной можно присвоить значение, указав ее в списке выбора инструкции SELECT. Значение переменной может являться константой, получаться

в результате вычисления выражения или работы функции или выбираться из таблицы. Чтобы присвоить значение переменной при помощи инструкции SET, необходимо указать ее имя и присваиваемое значение. Этот способ присвоения значений переменным является предпочтительным. Например :

```
DECLARE @FirstNameVariable nvarchar(50),
        @PostalCodeVariable nvarchar(15)

SET @FirstNameVariable = 'Amy'
SET @PostalCodeVariable = 'BA5 3HX'
```

Переменной можно присвоить значение, указав ее в списке выбора. Если список выбора ссылается на переменную, то ей должно быть присвоено скалярное значение, или инструкция SELECT должна возвращать только одну строку. Например:

```
DECLARE @EmpIDVariable int

SELECT @EmpIDVariable = MAX(EmployeeID) FROM Employee
```

Если инструкция SELECT возвращает более одной строки, и переменная ссылается на не скалярное выражение, ей присваивается значение, которое возвращается для выражения в последней строке результирующего набора.

Следует помнить, что, **для ASE версий младше 15**, если одна операция **select** задает значение нескольких переменных и одни и те же переменные присутствуют как слева, так и справа от знака равенства, то для присвоения всех значений будут использоваться те значения переменных, которые были непосредственно перед началом операции **select**. Например:

```
-- ASE младше 15.0
declare @i    int,
        @j    int

select @i = 1

select @i = @i + 1,  -- новое значение @i будет доступно после select
       @j = @i      -- используется старое значение @i
```

После выполнения приведенного участка кода значение переменной **@j** будет равно **1**, а не **2**, как можно было бы предположить. Если в переменную **@j** нужно занести уже увеличенное значение **@i**, вторую операцию **select** следует разделить на две.

Для ASE 15.0 и старше значения переменных изменяются сразу:

```
-- ASE 15.0 и старше
```

```
declare @i    int,  
        @j    int  
  
select @i = 1  
  
select @i = @i + 1,  -- новое значение @i доступно сразу  
       @j = @i      -- используется новое значение @i
```

Основные встроенные single-row функции

Использование встроенных функций значительно расширяет возможности базового оператора SELECT. В этом разделе рассматриваются так называемые single-row function (в дальнейшем просто функции), которые реализованы в Transact SQL.

Встроенная функция – это встроенный объект СУБД, реализующий определенный сервис

- Функция должна возвращать единственное значение
- Встроенная функция не может быть переопределена пользователем
- Функции не оперируют множеством значений

Функции могут использоваться для:

- Получения информации о системе
- Выполнении вычислений
- Модификации отдельных значений
- Преобразования типов

Синтаксис:

```
function_name( [{column | expression | const}  
                [, {column | expression | const} ...]] )
```

Аргументом функции может быть:

- Константа
- Переменная
- Имя столбца
- Выражение

Свойства функций:

- Может использоваться в предложениях SELECT, WHERE, ORDER BY
- Вызывается для каждой возвращаемой строки запроса
- Возвращает один результат на строку
- Тип возвращаемого значения может отличаться от типа аргумента
- Может принимать один и более аргументов
- Допускает вложенные вызовы

Встроенные функции можно разделить на следующие группы:

- Строковые: принимают строку в качестве одного из аргументов и возвращают или строку или число
- Математические (числовые): принимают и возвращают число
- Работы с датами: оперируют аргументами типа `datetime`
- Преобразования типа: конвертируют значение одного типа в другой тип
- Системные функции: предоставляют сервис общего назначения

Строковые функции

Функции преобразования регистра

lower(char_expr)	Преобразовывает все символы строки в нижний регистр. Возвращает символьное значение
upper(char_expr)	Преобразовывает все символы строки в верхний регистр. Возвращает символьное значение

Здесь **char_expr** - имя столбца символьного типа, переменная, константа или выражение типа char)

Пример:

```
SELECT "Фамилия"=lower(user_name1), "имя"=upper(user_name2) FROM susers
```

Сервисные функции

ascii(char_expr)	Возвращает ASCII код первого символа аргумента.
char(integer_expr)	Преобразовывает однобайтовое целочисленное значение в символьное значение. Значение integer_expr должно быть между 0 и 255
charindex(char_expr1, char_expr2)	Ищет первое вхождение char_expr1 в char_expr2 и возвращает целое число, представляющее начальную позицию (считая с 1). Если char_expr1 не найдено, возвращается 0 . Групповые символы не поддерживаются.
space(integer_expr)	Возвращает строку с обозначенном числом однобайтовых пробелов.
soundex(char_expr)	Возвращает четырехсимвольный soundex код для символьных строк.
char_length(char_expr)	Возвращает целое число, представляющее число символов в символьном выражении или текстовом значении. Для выражений с типом данных переменной длины char_length возвращает число символов. Для выражений с типом данных фиксированной длины возвращается размерность типа данных. <pre>declare @s1 varchar(10), @s2 char(10)</pre>

	<pre>select @s1 = 'AA', @s2 = 'AA' select char_lendth(@s1) -- 2, поскольку varchar select char_lendth(@s2) -- 10, поскольку char</pre>
reverse(char_expr)	Инвертирует аргумент («abcd» в «dcba»)
patindex("%pattern%", char_expr [using {chars bytes}])	Возвращает целое число, представляющее исходную позицию первого местонахождения шаблона в определенном символьном выражении. Если шаблон не найден, возвращается 0. По умолчанию, patindex возвращает смещение в символах. В шаблоне поддерживаются групповые символы, аналогичные используемым в операторе like. Групповой символ % должен предшествовать и следовать за шаблоном, за исключением поиска первых или последних символов строки.

Пример:

```
SELECT "ascii('Aa')" = ascii('Aa'),
       "char(ascii('A'))" = char(ascii('A')),
       "ascii('a')" = ascii('a'),
       "charindex('3', '123456')" = charindex('3', '123456')
```

```
ascii('Aa') char(ascii('A'))  ascii('a')  charindex('3', '123456')
-----
65              A              97              3
```

Функции манипулирования строками

ltrim(char_expr)	Удаляет начальные пробелы из символьного выражения. Если аргумент содержит только пробелы, возвращается NULL .
rtrim(char_expr)	Удаляет «хвостовые» пробелы из символьного выражения. Если аргумент содержит только пробелы, возвращается NULL .
right(expression, integer_expr)	Возвращает часть символьного значения, начинающуюся с заданной позиции, считая справа. Результат имеет тот же тип, что и аргумент.
substring(expression, start, length)	Возвращает часть строки. Start определяет символьную позицию, с которой подстрока начинается. length определяет число символов в результирующей строке.
replicate(char_expr, integer_expr)	Возвращает строку, составленную из аргумента, повторенного заданное число раз, до тех пор, пока длина результирующей строки не

	превысит 255.
stuff(char_expr1, start, length, char_expr2)	Удаляет length символов из char_expr1 , начиная с позиции start и в эту позицию вставляет char_expr2 . Для удаления без вставки в качестве char_expr2 можно использовать NULL (не '').

Пример:

```
SELECT x = substring("abcdef", 2, 3)
```

```
x
-----
bcd
```

```
SELECT Ex1 = stuff("abc", 2, 3, "xyz"),
       Ex2 = stuff("abcdef", 2, 3, null),
       Ex3 = stuff("abcdef", 2, 3, "")
```

```
Ex1    Ex2    Ex3
----    ----    ----
axyz   aef    a ef
```

Отдельно нужно отметить оператор **конкатенации** «+», который позволяет объединять строки:

```
SELECT "Обращение" = 'Уважаемый ' + user_name2 + ' ' + user_name3 + '!'
FROM susers
```

```
Обращение
-----
Уважаемый Иван Иванович!
```

Математические функции

Выполняют действия, обычно необходимые для математических расчетов.

abs(numeric)	Возвращает абсолютное значение данного выражения numeric . Результат имеет тот же самый тип и точность, что и аргумент.
ceiling(numeric)	Возвращает наименьшее целое, большее или равное аргументу numeric (округление "вверх"). Результат имеют тот же тип, что и аргумент.
floor(numeric)	Возвращает наибольшее целое, меньшее или равное аргументу numeric (округление "вниз"). Результат имеет тот же самый тип, что и

	аргумент.
round(numeric,integer)	Округляет аргумент numeric с точностью указанной integer . Положительное число integer определяет число значащих цифр справа от десятичной точки (округление до десятых, сотых и т.д.). Отрицательное число integer - число значащих цифр слева от десятичной точки.(округление до десятков, сотен и т.д.). Результат имеет тот же самый тип и точность, что и аргумент numeric .
Вне рассмотрения	acos(approx_numeric), asin(approx_numeric), atan(approx_numeric), atn2(approx_numeric), cos(approx_numeric), sin(approx_numeric), cot(approx_numeric), exp(approx_numeric), log(approx_numeric), log10(approx_numeric), rand([integer]) и т.д.

Функции работы с датами

Эти функции выполняют арифметические операции со значениями типа **smalldatetime** и **datetime**.

Формат хранения значений типа **datetime** – два целых числа (8 байт). Первые четыре байта сохраняют число дней до или после базовой даты (1.01.1900). Даты более ранние, чем 1.01.1753 не допускаются. Вторые четыре байта сохраняют время с точностью до 1/300 секунды.

Smalldatetime хранит данные с меньшей точностью, чем **datetime**. **Smalldatetime** хранится как целое (4 байта). В первых двух байтах хранится число дней после 1.01.1900, во вторых - число минут начиная с полночи. Диапазон возможных значений дат: 1.01.1900 - 6.06.2079, с точностью до 1 минуты.

Заданный по умолчанию формат отображения для дат - Mon dd yyyy hh:mm {AM | PM}
Например: Apr 15 2004 10:23AM.

Getdate()	Возвращает текущую дату и время
Datename(datepart, date)	Возвращает часть datetime или smalldatetime в виде строки
Datepart(datepart, date)	Возвращает часть datetime или smalldatetime как целое
Datediff(datepart, date1, date2)	Возвращает разницу между date2 и date1 приведенную к указанной в datepart единице измерения
Dateadd(datepart, number, date)	Возвращает дату, полученную в результате сложения заданных отрезков времени и date

Функции **datename**, **datepart**, **datediff** и **dateadd** в качестве параметра могут получать аббревиатуру **datepart** - Год, месяц, час, и так далее. Возможные значения **datepart**:

Date Part	Аббревиатура	Диапазон значений
year	yy	1753-9999
quarter	qq	1-4
month	mm	1-12
week	wk	1-54
day	dd	1-31
dayofyear	dy	1-366
weekday	dw	1-7
hour	hh	0-23
minute	mi	0-59
second	ss	0-59
millisecond	ms	0-999

Примеры использования:

```
-- Получение текущей даты, номера текущего месяца и дня, названия месяца
SELECT getdate(), datepart(month, getdate()),
       datepart(day, getdate()), datename(mm, '1971-12-15')
```

```
----- - -- -----
2013-01-21 11:02:33.796 1 21 December
```

```
-- Сложение и вычитание дат
SELECT datediff(year, '2003', '2004'),
       dateadd(dd, 16, '2013-01-01'),
       datediff(month, '1971-12-15', '1971-10-15')
```

```
- ----- --
1 2013-01-17 00:00:00.0 -2
```

Системные функции

Возвращают служебную информацию из базы данных. Большинство из них – быстрая альтернатива запросам из различных системных таблиц.

Некоторые системные функции :

Функция	Свойство
host_name ()	Возвращает ИД клиентского процесса.
host_id()	Возвращает имя компьютера клиентского процесса.
object_name(object_id [, database_id])	Возвращает имя объекта
object_id (object_name)	Возвращает ид объекта
(s)user_id([(server_)user_name])	Возвращает ИД пользователя (на сервере) в текущей БД.
(s)user_name([(server_)user_id])	Возвращает имя пользователя (на сервере) в текущей БД.
isnull(expr1, expr2)	Возвращает expr2, если expr1 равно NULL
tsequal(timestamp, timestamp2)	Сравнивает значения timestamp, чтобы предотвратить модификацию строки, которая изменилась, с момента последнего чтения timestamp.
col_length(object_name, column_name)	Возвращает длину столбца. Актуальную длину данных возвращает datalength(expression)
db_name ([database_id])	Возвращает имя базы данных
db_id ([database_name])	Возвращает ИД базы данных
datalength(expression)	Возвращает фактическую длину указанного столбца или строки в байтах. Возвращает NULL, если аргументом является NULL.

Примеры:

```
SELECT col_length('susers', 'user_name1'),
       datalength(user_name1),
       user_name1
FROM susers
```

		USER_NAME1
35	9	Звягинцев
35	7	Ефремов
35	8	Шевченко

Когда поле типа `char` определяется допускающим `NULL`, ASE сохраняет это поле как `varchar`. Для полей других типов возвращается их длина определенная при создании таблицы:

```
SELECT datalength(comments), datalength(object)
FROM sysattributes
WHERE comments IS NULL
```

```
-----
NULL                4
NULL                4
NULL                4
```

Функция **ISNULL** позволяет заменить значение `NULL` на более осмысленное значение. Тип результата равен типу первого аргумента, поэтому второй аргумент должен или допускать неявное преобразование, или быть явно преобразован к этому типу:

```
declare @d datetime,@i int,@s varchar
select isnull(@d,'2013-01-21'), isnull(@i,13),
       isnull(@s,'Nothing else matters')
```

```
-----      ---      -----
2013-01-21 00:00:00.0   13   Nothing else matters
```

Функции преобразования типов

В некоторых случаях, возникает необходимость использовать тип данных отличный от того, который планировал использовать разработчик. При этом ASE позволяет использовать некоторые типы данных вместо других, преобразовывая их в необходимый формат автоматически. Это называется неявное преобразование (**implicit conversion**) .

Например, если Вы сравниваете выражения с типами **char** и **datetime** или **smallint** и **int**, ASE автоматически преобразовывает один тип в другой.

Для других типов подобная попытка вызовет ошибку. В этом случае нужно вызывать встроенные функции преобразования типа. Это явное преобразование (**explicit conversion**). ASE обеспечивает три функции явного преобразования типа: **convert**, **inttohex** и **hextoint**.

И, наконец, некоторые типы данных не позволяют преобразовывать один в другой ни явно, ни неявно. Например, Вы не можете преобразовывать **smallint** данные в **datetime** и наоборот.

Матрица совместимости

Из/в	Int Smallint	Char Varchar	Datetime Smalldatetime	Decimal Numeric
Int Smallint	-	E	U	I
Char Varchar	E	-	I**	E
Datetime Smalldatetime	E	I	-	U
Decimal Numeric	I	E	U	- *

E – необходимо явное преобразование

I – допускается неявное преобразование

U – преобразование невозможно

* При преобразовании данных типа decimal(numeric) в данные того же типа, необходимо использовать явное преобразование, если при конвертации происходит потеря точности данных.

** Неявное преобразование возможно, если данные представлены в подходящем формате.

Примеры:

Необходимо явное преобразование

```
declare @a char(1)
declare @b int
select @a='1'
select @b=@a
```

Вывод:

ServerMessage: Number 257, Severity 16 Line 4:

Implicit conversion from datatype 'CHAR' to 'INT' is not allowed. Use the CONVERT function to run this query.

Неявное преобразование данных типа char в тип datetime возможно, если данные представлены в подходящем формате.

```
declare @d datetime, @s char(10)
select @s = '2013-01-01'
select @d=@s
select @d
```

Вывод:

```
-----
01.01.2013 0:00:00
```

При преобразовании данных типа decimal (numeric) в данные того же типа, необходимо использовать явное преобразование, если при конвертации происходит потеря точности данных.

При выполнении произойдет ошибка:

```
declare @a decimal(5,2)
declare @b decimal(4,1)
select @a=15.23
select @b=@a          -- Line4: Truncation error occurred.
```

Выполнение без ошибок:

```
declare @a decimal(5,2)
```

```
declare @b decimal(4,1)
select @a=15.23
select @b = convert(decimal(4,1),@a)
```

Явное преобразование типов

Проводится функцией **convert**. Синтаксис:

```
convert(datetype, expression, [style])
```

convert – основная функция, которая преобразовывает данные одного типа в другой. При преобразовании из **char** в **datetime** и наоборот позволяет указать формат представления данных.

Некоторые типы данных при определении предполагают наличие длины или точности и масштаба. Если длина не определена, то длина по умолчанию для символьных и двоичных данных - **1** символ. Если для типов **numeric** или **decimal** не определена разрядность или точность, по умолчанию используются значения **18** и **0** соответственно.

Примеры:

Преобразование char -> int

```
declare @a char(5), @b int
select @a='537'
select @b=convert(int, @a)
select @b
```

Преобразование int -> char

```
declare @a char(5), @b int
select @b=537
select @a=convert(char, @b)
select @a
```

Параметр **style** функции **convert** предоставляет ряд форматов представления данных при преобразовании **datetime/smalldatetime** к **char** или **varchar**. Каждый формат имеет свой определенный номер, который передается на вход функции **convert**.

Год может отображаться в 2-символьном или 4-символьном виде (чтобы получить 4-символьное представление нужно прибавить 100 к соответствующему номеру 2-символьного формата).

Формат представления данных

«Короткий» (yy)	«Длинный» (yyyy)	Стандарт	Представление
-	0 or 100	Default	Mon dd yyyy hh:mm AM (or PM)
1	101	USA	mm/dd/yy
2	102	SQL	yy.mm.dd
3	103	English/French	dd/mm/yy
4	104	German	dd.mm.yy
5	105	-	dd-mm-yy
6	106	-	dd mon yy
7	107	-	mon dd, yy
8	108	-	hh:mm:ss
-	9 or 109	Default + millisecond	Mon dd yyyy hh:mm:sss AM (or PM)
10	110	USA	mm-dd-yy
11	111	Japan	yy/mm/dd
12	112	ISO	yymmdd

Пример:

```
select convert(char,getdate(),112),
       convert(char(12),getdate(),3),
       convert(char,getdate())
```

```
-----
20130121    21/01/13    Jan 21 2013 10:52AM
```


Оператор CASE

Реализует условные SQL выражения (логика IF-THEN-ELSE), может использоваться в любом месте, в котором используются выражения.

Команда CASE реализует ту же функциональность, что и условный оператор в других языках программирования. Особая ценность и сходство с single-row функциями оператора CASE заключается в том, что он может использоваться в операторах SELECT, UPDATE, DELETE и вычисляется для **каждой** возвращаемой строки.

Оператор CASE имеет два формата:

- простое выражение CASE для определения результата сравнивает выражение с набором простых выражений;
- поисковое выражение CASE для определения результата вычисляет набор логических выражений.

Оба формата поддерживают дополнительный аргумент **ELSE**.

Синтаксис:

```
CASE выражение
  WHEN значение_1 THEN значение_2
  [WHEN значение_n THEN значение_n ...]
  [ELSE] значение_default
END
```

Здесь сравнивается **значение_1** с результатом **выражение** и если результат данного сравнения истина(true), то CASE возвращает **значение_2**. Таких пар значений можно сделать много. Также можно задать значение в операторе ELSE, если не выполняться ни одно из условий.

И вторая форма записи оператора CASE, которая позволяет сравнивать не одно выражение с несколькими значениями, а задавать условия для каждого случая:

```
CASE
  WHEN условие_1 THEN значение_1
  [WHEN условие_n THEN значение_n ...]
  [ELSE] значение_default
END
```

Если значение по умолчанию опущено, Case возвращает значение NULL, демонстрируя тот факт,

что вычисляемое выражение не совпало ни с одним из предложенных вариантов.

Примеры:

При использовании в инструкции SELECT простое выражение CASE позволяет выполнить только проверку на равенство. Другие проверки не выполняются. В следующем примере выражение CASE используется для изменения способа отображения категорий линейки продуктов с целью сделать их более понятными (**ProductLine** - поле в таблице **Product**) :

```
SELECT ProductNumber,
       Category = CASE ProductLine
                   WHEN 'R' THEN 'Road'
                   WHEN 'M' THEN 'Mountain'
                   WHEN 'T' THEN 'Touring'
                   WHEN 'S' THEN 'Other sale items'
                   ELSE 'Not for sale'
       END,
       Name
FROM Production..Product
```

При использовании в инструкции SELECT поисковое выражение CASE позволяет заменять значения в результирующем наборе в зависимости от результатов сравнения. В следующем примере отображается список цен в виде текстового комментария, основанного на диапазоне цен для продукта:

```
SELECT Title,
       CASE
           WHEN price IS NULL THEN 'Unpriced'
           WHEN price < 10 THEN 'Under $10'
           WHEN price >= 50 and price < 250 THEN 'Under $250'
           WHEN price >= 250 and price < 1000 THEN 'Under $1000'
           ELSE 'Over $1000'
       END as 'Price Range'
FROM titles
```

Использование выражения CASE в предложении ORDER BY

В следующем примере выражение CASE используется в предложении ORDER BY, чтобы определить порядок сортировки строк на основе значения заданного столбца таблицы. Здесь вычисляется значение столбца SalariedFlag таблицы Employee. Сотрудники, для которых столбец SalariedFlag имеет значение 1, возвращаются в порядке EmployeeID (по убыванию). Сотрудники, для которых столбец SalariedFlag имеет значение 0, возвращаются в порядке EmployeeID (по возрастанию). Для примера используются обе формы оператора CASE :

```
SELECT EmployeeID, SalariedFlag
FROM Employee
ORDER BY CASE SalariedFlag WHEN 1 THEN EmployeeID END DESC,
CASE WHEN SalariedFlag = 0 THEN EmployeeID END
```

Использование выражения CASE в инструкции UPDATE

В следующем примере выражение CASE используется в инструкции UPDATE, чтобы определить значение, установленное в столбце VacationHours для сотрудников, у которых столбец SalariedFlag имеет значение 0. Если при вычитании 10 часов из VacationHours получается отрицательное значение, VacationHours увеличивается на 40 часов. В противном случае значение VacationHours увеличивается на 20 часов:

```
UPDATE Employee
SET VacationHours = CASE
    WHEN ((VacationHours - 10.00) < 0)
    THEN VacationHours + 40
    ELSE (VacationHours + 20.00)
END
WHERE VacationHours NOT IS NULL
```

Группировка данных

Применение предложения **GROUP BY** позволяет разделить таблицу на наборы данных. В контексте оператора **SELECT** предложение **GROUP BY** имеет следующий вид:

```
SELECT [column,] aggregate_function (expression)
      FROM table [, table_2, ..., table_N]
[WHERE (N-1) join_conditions ]
[GROUP BY groupby_expression]
[ORDER BY column [, column]]
```

где **groupby_expression** – определяет список столбцов или выражений, которые определяют базис для создания групп данных.

Предложение **GROUP BY** делит вывод запроса на группы. Группы можно создавать по одному или нескольким столбцам таблицы, или по результатами вычисленных выражений. Агрегирующие функции вычисляются для каждой подгруппы. Максимальное число столбцов или выражений, которые можно использовать в предложении **GROUP BY** - 16.

Понятие группировки неразрывно связано с понятием агрегатных функций.

Агрегатные функции

Агрегатные функции выполняют вычисление на наборе значений и возвращают одиночное значение. Агрегатные функции, за исключением **COUNT**, не учитывают значения **NULL**. Агрегатные функции часто используются в выражении **GROUP BY** инструкции **SELECT** (будет рассмотрено далее).

Все агрегатные функции являются детерминированными. Это означает, что агрегатные функции возвращают одну и ту же величину при каждом их вызове на одном и том же наборе входных значений.

Агрегатные функции могут быть использованы в качестве выражений только в следующих случаях:

- Список выбора инструкции **SELECT** (вложенный или внешний запрос).
- Предложение **HAVING**.

<code>avg ([DISTINCT ALL] column)</code>	Возвращает среднее значение по указанному столбцу
<code>count (* [DISTINCT ALL] column)</code>	Возвращает количество элементов в выборке или в группе, определяемой указанным столбцом
<code>sum ([DISTINCT ALL] column)</code>	Возвращает сумму значений указанного столбца
<code>max (column)</code>	Возвращает максимальное значение в столбце
<code>min (column)</code>	Возвращает минимальное значение в столбце

Ключевое слово **DISTINCT** позволяет игнорировать повторные значения в столбце, **ALL** обрабатывает все значения в столбце (по умолчанию), * позволяет включить в обработку поля со значением **NULL**.

Ниже приведен пример использования агрегатных функций в качестве выбираемых данных. Если агрегатная функция используется в выборке без **GROUP BY**, то она применяется ко всем записям выборки, иначе для каждой группы в отдельности. И в любом случае в перечислении select нельзя смешивать групповые столбцы с не групповыми.

```
-- статистические данные по всем месяцам
```

```

select count(*) as "число записей",
       max(t.summ) as "макс. сумма",
       min(t.summ) as "мин. сумма",
       avg(t.summ) as "средняя сумма",
       sum(t.summ) as "общая сумма"
from bills t

-- статистические данные по каждому месяцу
select t.d as "месяц", count(1) as "число записей",
       max(t.summ) as "макс. сумма",
       min(t.summ) as "мин. сумма",
       avg(t.summ) as "средняя сумма",
       sum(t.summ) as "общая сумма"
from bills t
group by t.d

```

Особенности использования

- Агрегирующие функции нельзя использовать в предложении **WHERE**. Если предложение **SELECT** включает предложение **WHERE**, но не включает **GROUP BY**, функция агрегата производит единственное значение для подмножества строк, называемых в этом случае **скалярным агрегатом**. Например, данный запрос вернет единственный результат:

```
SELECT SUM(size) FROM sysusages
```

- Если имеется предложение **GROUP BY**, то это позволяет разделить таблицу на наборы данных, а агрегирующие функции вычисляют для каждого набора итоговое значение. Эти значения называются **агрегирующим вектором**. Например, данный запрос вернет набор значений:

```
SELECT dbid, SUM(size) FROM sysusages GROUP BY dbid
```

- Агрегирующие функции всегда имеют аргументы. В качестве аргументов обычно используются столбцы таблиц, но также допускаются константы, функции и любые их комбинации с арифметическими операторами.
- Со всеми агрегирующими функциями, кроме **COUNT(*)**, можно использовать ключевое слово **DISTINCT**, что ограничивает набор аргументов неповторяющимися значениями. Хотя не имеет никакого смысла использовать его с функциями, т.к. получаемые значения в обоих случаях будут одинаковы.
- Все агрегирующие функции, за исключением **COUNT(*)**, игнорируют **NULL** значения. Для

замены **NULL** значений, можно использовать функцию **ISNULL**.

- При использовании **GROUP BY** Sybase неявно сортирует результаты запроса в возрастающем порядке.
- Аргументами для агрегирующих функций **AVG** и **SUM** могут выступать столбцы, содержащие только числовые данные.
- Функции **MAX** и **MIN**, **COUNT** нельзя использовать со столбцами типа **bit**.
- С полями типа **text** и **image** нельзя использовать агрегатные функции, кроме **COUNT(*)**.
- С функциями **AVG**, **COUNT**, **MAX**, **MIN**, **SUM** можно использовать ключевое слово **DISTINCT**, помещая его перед аргументом. В этом случае при вычислении результата не учитываются повторяющиеся строки. Использование **DISTINCT** в списке выбора оператора **SELECT** и в агрегирующей функции приводит к разным результатам:

```
SELECT DISTINCT COUNT(size), COUNT( DISTINCT size) FROM sysusages
```

Вывод:

```
-----
6          5
```

В примере сначала выполняется функция, которая возвращает в результате одну строку, а затем к ее результатам применяется **DISTINCT**, и второй оператор **COUNT** возвращает число уникальных идентификаторов.

Предложение **GROUP BY**

SQL-стандарт для **GROUP BY** требует чтобы:

- столбцы, которые используются в списке выбора должны быть указаны в предложении **GROUP BY**, или они должны быть параметрами агрегирующих функций.
- **GROUP BY** может только содержать имена столбцов из списке выбора, но не столбцы, служащие аргументами для агрегирующих функций.

Вводя следующие расширения, Transact SQL ослабляет эти требования, допуская наличие любого допустимого столбца (константы, выражения) или в группе, или списке выбора, независимо от того используются они в агрегатах или нет:

- столбцы в списке выбора не ограничены группируемыми столбцами и столбцами, используемыми с векторными агрегатами.
- столбцы, определяемые **GROUP BY**, не ограничены столбцами из списка выбора, не

являющимися аргументами агрегирующих функций.

Общие рекомендации по использованию предложения **GROUP BY**:

- Используя предложение **WHERE**, вы можете ограничить количество данных, которые будут подвергнуты групповой обработке
- Желательно включать в **GROUP BY** все столбцы, указанные в списке выбора и на являющиеся аргументами агрегирующих функций
- В предложении **GROUP BY** нельзя использовать псевдонимы (alias) столбцов.

Пример :

```
SELECT dbid, sum(size)
  FROM sysusages
 WHERE crdate > '2005-01-01'
GROUP BY dbid
```

Порядок вычисления:

1. Список выбора оператора **SELECT** определяет выбираемые столбцы: **dbid**, сумму размеров всех девайсов для группы, указанной в предложении **GROUP BY**
2. Предложение **FROM** определяет таблицу(ы) доступа (**sysusages**)
3. Выполняется предложение **WHERE** оператора **SELECT**.
4. Формируются уникальные группы значений столбцов, указанных в предложении **GROUP BY** во временной таблице.
5. Производится объединение списка выбора исходной и временной таблиц с вычислением агрегатов, если они заданы, для групп из п.2.

Путем группировки одновременно по нескольким элементам можно создавать группы внутри групп. Разделяя элементы, по которым будет проводиться группировка, с помощью запятых можно разбить большие группы на подгруппы:

```
SELECT dbid, dbname, sum(size)
  FROM sysusages
GROUP BY dbid, dbname
```

Сначала строки таблицы разделяются по идентификаторам **dbid**, а затем каждая полученная группа разделяется по названиям объекта **dbname**, чьим владельцем является идентификатор из родительской группы. Для каждого названия высчитывается значение суммы поля **size**.

GROUP BY ALL

Ключевое слово **ALL** в предложении **GROUP BY** – расширение Transact SQL. Его использование имеет смысл только в том случае, если оператор **SELECT**, кроме предложения **GROUP BY**, включает также предложение **WHERE**.

Если используется **ALL**, в результат запроса включаются все группы, полученные предложением **GROUP BY**, даже если некоторые из них не удовлетворяют условиям **WHERE**. Иначе, в результат запроса включаются только те группы, для которых условие **WHERE** верно.

Запрос

```
SELECT program_name, sum(physical_io)
FROM sysprocesses
WHERE physical_io > 300
GROUP BY program_name
```

вернет

program_name	
-----	-----
Sybridge	450
SQL_Advantage	7000

а запрос

```
SELECT program_name, sum(physical_io)
FROM sysprocesses
WHERE physical_io > 300
GROUP BY ALL program_name
```

вернет

program_name	
-----	-----
Sybridge	450
SQL_Advantage	7000
SQL_CENTRAL	NULL

NULL и GROUP BY

Если столбец, по которому выполняется группировка, содержит значение **NULL**, то соответствующие строки составят в результатах запроса собственную группу.

Несмотря на то, что значения **NULL** бессмысленно сравнивать между собой, в эту группу будут собраны все строки, для которых значение столбца группировки равно **NULL** :

```
SELECT clientapplname, count(*), count(clientapplName)
FROM sysprocesses
GROUP BY clientapplname
```

Вывод :

clientapplname		
NULL	2	0
Sybridge	2	2
Sybase_Central	1	1

По умолчанию, результирующие строки отсортированы в возрастающем порядке согласно списку **GROUP BY**. Чтобы изменить порядок вывода по умолчанию нужно использовать предложение **ORDER BY**.

При этом действуют все соглашения принятые для предложения **ORDER BY** при использовании его с оператором **SELECT**.

Пример:

```
SELECT clientapplname, count(*), count(clientapplName)
FROM sysprocesses
GROUP BY clientapplname
ORDER BY 3, 1
```

Вывод:

clientapplname		
NULL	2	0
Sybase_Central	1	1
Sybridge	2	2

Предложение HAVING

Для отбора групп по условию существует оператор **HAVING**. Предложение **HAVING** можно использовать только в инструкции **SELECT**. Предложение **HAVING** обычно используется в предложении **GROUP BY**.

Если в списке выбора имеются агрегирующие функции, предложение **WHERE** выполняется перед ними, тогда как предложение **HAVING** применяется ко всему запросу в целом, после разбиения на группы и вычисления агрегатов.

Когда **GROUP BY** не используется, предложение **HAVING** работает так же, как и предложение **WHERE**. С точки зрения условного выражения, **HAVING** и **WHERE** идентичны, отличие состоит лишь в том, что в условии предложения **WHERE** не могут находиться агрегирующие функции.

Синтаксис:

```
SELECT [column,] aggregate_function (expression)
      FROM table [, table_2, ..., table_N]
[WHERE (N-1) join_conditions ]
[GROUP BY groupby_expression]
[HAVING group_condition]
[ORDER BY column [, column]]
```

Последовательность вычисления результатов запроса:

1. Исключаются строки не удовлетворяющие условиям предложения **WHERE**
2. Оставшиеся строки объединяются в наборы согласно выражению **GROUP BY**
3. Для каждого набора вычисляются агрегаты
4. Из наборов, удовлетворяющих условиям **HAVING**, формируется результат

Если в **HAVING** есть несколько условий, они объединяются с помощью логических операторов **AND**, **OR** и **NOT**.

Примеры использования:

Подсчет средней стоимости заказа для каждого служащего из числа тех, у которых общая стоимость заказов превышает \$300

```
SELECT REP, AVG(AMOUNT)
      FROM ORDERS
GROUP BY REP
HAVING SUM(AMOUNT) > 300
```

Использование в **HAVING выражений, не представленных в списке выбора.**

Запрос определяет средний размер для тех баз данных, чей суммарный размер превосходит 10000 страниц. Замена **HAVING** предложением **WHERE** вызвала бы в данном случае синтаксическую ошибку:

```
SELECT dbid, avg(size)
```

```
FROM sysusages
GROUP BY dbid
HAVING sum(size) > 10000
```

Использование HAVING без GROUP BY.

В этом случае, действие **HAVING** аналогично действию **WHERE**, т.к. условия **HAVING** накладываются на все строки таблицы, которая трактуется как единая группа. Кроме того, в условии возможно использование агрегирующих функций:

```
SELECT dbid, crdate, size
FROM sysusages
WHERE crdate < '2005-01-01'
HAVING size > avg(size)
```

Подзапросы в предложении HAVING

Вы можете также использовать подзапросы внутри предложения **HAVING**. Компиляция и выполнение подобных запросов осуществляется традиционно: сначала выполняется внутренний подзапрос, и его результаты подставляются в предложение **HAVING** основного запроса. Эти подзапросы могут использовать свои собственные агрегатные функции, если они не производят нескольких значений, или использовать **GROUP BY** или **HAVING**.

Следующий запрос является примером этого :

```
SELECT rating, COUNT (DISTINCT cnum)
FROM Customer
GROUP BY rating
HAVING rating > (SELECT AVG (rating)
                  FROM Customers
                  WHERE city = 'San Jose')
```

Эта команда получает средний рейтинг заказчиков в San Jose и использует это значение при отборе групп по условию: рейтинги должны быть выше полученного. Так как имеются различные рейтинги, они выводятся с количеством номеров заказчиков, которые имеют этот рейтинг.

Подзапросы, которые используют предложения **GROUP BY** должны обеспечивать механизм, контролирующий возврат соответствующего внешнему запросу количества значений.

Примером такого механизма может служить использование предложения **HAVING**:

```
SELECT suid, program_name
FROM sysprocesses
WHERE time_blocked > (SELECT avg(time_blocked)
                      FROM sysprocesses)
```

```
WHERE status = 'lock_sleep'  
GROUP BY program_name  
HAVING program_name = 'RZPS')
```